

PDTN Phase B - Σημειώσεις Εξέτασης

Ελληνική μεταφρασμένη και προσαρμοσμένη έκδοση

Βασισμένο στο αγγλικό PDF που ανέβηκε στο chat. Σε αυτή την έκδοση το επεξηγηματικό κείμενο έχει αποδοθεί στα ελληνικά, ενώ τα snippets κώδικα παραμένουν σε Python ώστε να είναι άμεσα χρήσιμα στον διαγωνισμό.

Στόχος αυτής της έκδοσης δεν είναι η απόλυτα τυπογραφική αντιγραφή του αρχικού αρχείου, αλλά μια καθαρή ελληνική έκδοση με την ίδια λογική δομή: γρήγορη πλοήγηση, έτοιμα templates, βασικά debug checks και recipes ανά τύπο προβλήματος.

Κανόνας χρήσης στον διαγωνισμό: baseline πρώτα, submit γρήγορα, μετά μόνο στοχευμένα tweaks.

Περιεχόμενα

- 0. Πώς να χρησιμοποιείς αυτές τις σημειώσεις
- 1. Δέντρο απόφασης προβλήματος - μοτίβου
- 2. Python survival: NumPy, pandas, matplotlib, I/O
- 3. sklearn cookbook
- 4. Data handling
- 5. PyTorch minimum
- 6. Computer vision
- 7. NLP και embeddings
- 8. Audio
- 9. Problem recipes
- 10. Optimization & solvers
- 11. Debugging & gotchas

0. Πώς να χρησιμοποιείς αυτές τις σημειώσεις

Περιβάλλον σύμφωνα με το αρχικό PDF: Ubuntu 24.04 LTS, Python 3.12, conda env ml_competition_env, με numpy/pandas/scipy/scikit-learn, torch/torchvision/torchaudio, xgboost/lightgbm/catboost, transformers, opencv, Pillow, albumentations, matplotlib, openpyxl, cvxpy, pulp, ortools, z3 και άλλα χρήσιμα εργαλεία.

- Υπό πίεση χρόνου, ξεκίνα από τα recipes. Μην διαβάζεις πρώτα όλη τη θεωρία.
- Διάβασε όλη την εκφώνηση δύο φορές και αναγνώρισε: metric, τύπο output, τύπο δεδομένων, περιορισμούς.
- Βρες ποιο μοτίβο ταιριάζει: embeddings, tabular CSV, raw text, raw image, fixed small neural net, sequence/time series, combinatorial optimization.
- Στήσε baseline ακριβώς όπως περιγράφεται. Κάνε submit νωρίς. Μην σπαταλάς 45 λεπτά πριν από το πρώτο submission.
- Χρησιμοποίησε το LLM μόνο για έννοιες και έλεγχο κατανόησης. Τον κώδικα πρέπει να τον έχεις στις σημειώσεις σου.
- Αν κολλήσεις: print shapes, print dtypes, κοίτα metric/loss, έλεγξε leakage, έλεγξε αν το target dtype ταιριάζει με τη loss.

1. Δέντρο απόφασης προβλήματος - μοτίβου

Τα πρώτα 5 λεπτά: διάβασε όλη την εκφώνηση, γράψε metric, αναγνώρισε output/data type, βρες recipe, υλοποίησε baseline, κάνε submit.

- Αν σου δίνουν έτοιμα embeddings: φέρσου στα vectors σαν κανονικά features. Για regression με MAE, baseline = `LGBMRegressor(objective="mae")`. Για classification, baseline = `LogisticRegression(max_iter=2000)`.
- Αν σου δίνουν fixed μικρό neural net για training: αν το input είναι συντεταγμένες (x, y) και το output χρώμα/ένταση, σκέψου Fourier positional encoding και Adam + cosine LR.
- Αν έχεις word embeddings και παιχνίδι / matching (τύπου Codenames): brute force σε vocabulary με cosine similarity.
- Αν έχεις raw text και πρέπει εσύ να φτιάξεις features: ξεκίνα με TF-IDF + LogisticRegression, μετά δοκίμασε pretrained encoder.
- Αν έχεις raw images: για μικρό dataset ξεκίνα με transfer learning (π.χ. resnet18). Για πολύ μικρά datasets πάγωσε σχεδόν όλο το backbone και βάλε ισχυρό augmentation.
- Αν έχεις CSV με αριθμητικές και κατηγορικές στήλες: LightGBM με category dtype, χωρίς scaling, χωρίς one-hot σαν baseline.
- Αν έχεις unlabeled data: KMeans + silhouette για clustering, PCA/UMAP για visualization.
- Αν έχεις sequence / time series: sliding windows + LightGBM είναι φθηνό και ισχυρό baseline.
- Αν έχεις discrete combinatorial problem: πιθανόν χρειάζεσαι solver και όχι ML μοντέλο.
- MAE -> median, MSE/RMSE -> mean, accuracy -> cross-entropy + argmax, F1 -> weighted cross-entropy + threshold tuning.

2. Python survival

2.1 Emergency card

- print και f-strings για debug και formatting.
- Λίστες: append, extend, slicing, reverse, list comprehensions.
- Λεξικά: d[key], d.get(key, default), d.items().
- Loops: range, enumerate, zip, break, continue.
- Strings: lower, strip, split, join, replace.
- Συνηθισμένα errors: IndentationError, NameError, TypeError, KeyError, IndexError, ValueError.

```
print("plain")
print(x, y)
print(f"x={x}, y={y:.3f}")

a = [1, 2, 3]
a.append(4)
a.extend([5, 6])
a[0], a[-1]
a[1:4], a[:2], a[::-1]

d = {"a": 1, "b": 2}
d["c"] = 3
d.get("x", 0)

for i in range(10):
    if i % 2 == 0:
        continue
    if i > 7:
        break
    print(i)

for i, x in enumerate(a):
    print(i, x)

for x, y in zip(a, b):
    print(x, y)
```

2.2 NumPy essentials

- NumPy = βασικό νόμισμα για sklearn/PyTorch.
- Θυμήσου reshape, axis, broadcasting, boolean masks, reductions και dtype checks.
- Πρόσεχε αν ο πίνακας είναι int ή float. Το NaN δεν ζει σε integer array.

```
import numpy as np

a = np.arange(12)
a = a.reshape(3, 4)
a[:, 1]
a[1:3]
mask = a > 5
a[mask] = 0

b = np.array([1, 2, 3, 4])
a + b
a.sum(axis=0)
a.mean(axis=1)

np.clip(a, 0, 1)
np.round(a, 2)
np.isnan(a)
np.save("arr.npy", a)
a = np.load("arr.npy")
```

2.3 pandas essentials

- read_csv, head, shape, dtypes, info, describe.
- Πρόσβαση σε στήλες με df["col"] ή df[["a", "b"]].
- Φιλτράρισμα γραμμών με loc / iloc και συνθήκες.

- groupby, merge, concat, fillna, astype, to_datetime.
- Όπου γίνεται, προτίμησε vectorized πράξεις αντί για apply(axis=1).

```
import pandas as pd
import numpy as np

df = pd.read_csv("train.csv")
df.head()
df.shape
df.dtypes
df["price_per_m2"] = df["price"] / df["area"]

df = df[df["price"] > 100_000]
df = df.fillna({"price": df["price"].median(),
               "city": "unknown"})
df["date"] = pd.to_datetime(df["date"])
df["year"] = df["date"].dt.year

agg = df.groupby("city")["price"].agg(["mean", "std", "count"])
merged = pd.merge(left, right, on="user_id", how="left")

X = df[FEATURES].to_numpy()
y = df["label"].values
```

2.4 Matplotlib, I/O, timing και reproducibility

- Plot early, plot often. Πριν πειράξεις hyperparameters, κοίτα τα δεδομένα.
- Χρήσιμα plots: loss curves, scatter, histogram, confusion matrix, image batches.
- Pathlib για paths, json/pickle/joblib για I/O, tqdm για progress bars.
- Σπείρε random state σε random / numpy / torch.

```
import matplotlib.pyplot as plt
plt.figure(figsize=(6, 4))
plt.plot(x, y, label="train")
plt.plot(x, y2, label="val", linestyle="--")
plt.xlabel("epoch")
plt.ylabel("loss")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()

from pathlib import Path
root = Path("data")
for f in root.glob("*.csv"):
    print(f.name)

import random, numpy as np, torch
def seed_all(s=42):
    random.seed(s)
    np.random.seed(s)
    torch.manual_seed(s)
    torch.cuda.manual_seed_all(s)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

3. sklearn cookbook

Κάθε estimator ακολουθεί το ίδιο μοτίβο: κατασκευή με hyperparameters, fit στο train, predict στο validation/test. Για classifiers συχνά χρειαζόμαστε και predict_proba.

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.metrics import mean_absolute_error

Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.15, random_state=0)

pipe = make_pipeline(StandardScaler(), Ridge(alpha=1.0))
pipe.fit(Xtr, ytr)
pred = pipe.predict(Xva)
print("val MAE:", mean_absolute_error(yva, pred))
```

- Μοντέλα που θέλουν scaling: linear/logistic regression, Ridge/Lasso, SVM, KNN, KMeans, PCA, MLP και γενικά distance-based μοντέλα.
- Μοντέλα που δεν το χρειάζονται: decision trees, random forests, gradient boosting, LightGBM, XGBoost, CatBoost.
- LogisticRegression: πρόσχε ότι χρησιμοποιεί C (inverse regularization), ενώ Ridge/Lasso χρησιμοποιούν alpha.

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet, LogisticRegression

m1 = LinearRegression().fit(Xtr, ytr)
m2 = Ridge(alpha=1.0).fit(Xtr, ytr)
m3 = Lasso(alpha=0.01, max_iter=10000).fit(Xtr, ytr)
m4 = ElasticNet(alpha=0.01, l1_ratio=0.5, max_iter=10000).fit(Xtr, ytr)

clf = LogisticRegression(
    C=1.0,
    max_iter=2000,
    class_weight="balanced",
    n_jobs=-1,
    solver="lbfgs"
).fit(Xtr, ytr)

pred = clf.predict(Xva)
proba = clf.predict_proba(Xva)
```

- KNN: scale πάντα. Για embeddings συχνά metric="cosine".
- DecisionTree: max_depth και min_samples_leaf για overfitting control.
- RandomForest: ισχυρό baseline, αλλά συχνά LightGBM είναι γρηγορότερο και δυνατότερο.
- LightGBM: default ισχυρή επιλογή για tabular και embeddings.
- SVM: linear για sparse text, rbf για dense features.
- MLPClassifier/Regressor: χρήσιμο για dense embeddings, όχι τόσο για μικρό tabular.
- Pipeline = αποφυγή leakage. CV για σταθερό estimate, αλλά όχι grid search στην πρώτη ώρα του διαγωνισμού.

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from lightgbm import LGBMRegressor

knn = KNeighborsRegressor(
    n_neighbors=15,
    weights="distance",
    metric="cosine",
    n_jobs=-1
).fit(Xtr, ytr)

tree = DecisionTreeClassifier(
    max_depth=8,
    min_samples_leaf=10,
    class_weight="balanced",
    random_state=0
```

```
).fit(Xtr, ytr)

rf = RandomForestClassifier(
    n_estimators=500,
    min_samples_leaf=2,
    n_jobs=-1,
    random_state=0
).fit(Xtr, ytr)

lgbm = LGBMRegressor(
    n_estimators=2000,
    learning_rate=0.03,
    num_leaves=63,
    min_child_samples=20,
    objective="mae",
    verbose=-1
).fit(Xtr, ytr, eval_set=[(Xva, yva)])
```

4. Data handling

Train/val/test split σωστά, αλλιώς όλο το score είναι ψεύτικο. Για classification χρησιμοποίησε stratify=y. Για time series μην κάνεις shuffle. Για group-based data μην αφήνεις την ίδια οντότητα να μπει και στα δύο splits.

- fit στο train, transform σε val/test - ποτέ fit ξανά στο val/test.
- Για cosine similarity χρήσιμο είναι normalize(X, norm="l2", axis=1).
- Για neural nets σε regression συχνά βοηθάει να scaleάρεις και το target.
- Μην κάνεις scaling σε tree-based μοντέλα.
- One-hot για linear/SVM/neural nets. LightGBM/CatBoost χειρίζονται categoricals και NaNs καλύτερα.
- Target encoding μόνο με τεράστια προσοχή γιατί κάνει leakage εύκολα.
- Image augmentation μόνο στο training pipeline.
- Text cleaning: lower, remove URLs/punctuation, normalize whitespace, μετά TF-IDF ή transformer tokenization.

```
from sklearn.model_selection import train_test_split, GroupShuffleSplit
```

```
Xtr, Xva, ytr, yva = train_test_split(  
    X, y, test_size=0.15, random_state=0, stratify=y  
)
```

```
gss = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=0)  
tr_idx, va_idx = next(gss.split(X, y, groups=user_ids))  
Xtr, Xva = X[tr_idx], X[va_idx]  
ytr, yva = y[tr_idx], y[va_idx]
```

```
from sklearn.preprocessing import StandardScaler, normalize
```

```
sc = StandardScaler()  
Xtr_s = sc.fit_transform(Xtr)  
Xva_s = sc.transform(Xva)  
Xte_s = sc.transform(Xte)  
  
Xn = normalize(X, norm="l2", axis=1)  
  
y_mean, y_std = ytr.mean(), ytr.std()  
ytr_s = (ytr - y_mean) / y_std  
yva_s = (yva - y_mean) / y_std
```

```
import pandas as pd  
from sklearn.impute import SimpleImputer, KNNImputer  
  
X_oh = pd.get_dummies(df, columns=["city", "color"], dtype=float)  
X_te = pd.get_dummies(df_te, columns=["city", "color"], dtype=float)  
X_te = X_te.reindex(columns=X_oh.columns, fill_value=0)  
  
df["price"] = df["price"].fillna(df["price"].median())  
df["city"] = df["city"].fillna("unknown")  
  
num_imp = SimpleImputer(strategy="median")  
X_num = num_imp.fit_transform(X_num)  
  
knn_imp = KNNImputer(n_neighbors=5)  
X_num = knn_imp.fit_transform(X_num)
```


5. PyTorch minimum

Αυτό είναι το βασικό template training loop που πρέπει να ξέρεις απ' έξω. Αλλάζουν το model, η loss και οι loaders - όχι η λογική.

```
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader

device = "cuda" if torch.cuda.is_available() else "cpu"

Xtr_np = np.load("X_train.npy").astype(np.float32)
ytr_np = np.load("y_train.npy").astype(np.float32)
Xva_np = np.load("X_val.npy").astype(np.float32)
yva_np = np.load("y_val.npy").astype(np.float32)

Xtr = torch.from_numpy(Xtr_np)
ytr = torch.from_numpy(ytr_np)
Xva = torch.from_numpy(Xva_np)
yva = torch.from_numpy(yva_np)

train_ds = TensorDataset(Xtr, ytr)
val_ds = TensorDataset(Xva, yva)

train_loader = DataLoader(train_ds, batch_size=128, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=256, shuffle=False)

in_dim = Xtr.shape[1]
out_dim = 1

model = nn.Sequential(
    nn.Linear(in_dim, 256), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.2),
    nn.Linear(128, out_dim),
).to(device)

EPOCHS = 30
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=EPOCHS)
loss_fn = nn.MSELoss()

best_val = float("inf")
for epoch in range(EPOCHS):
    model.train()
    tr_losses = []
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)
        pred = model(x).squeeze(-1)
        loss = loss_fn(pred, y)

        opt.zero_grad()
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()

    tr_losses.append(loss.item())

    sched.step()

    model.eval()
    va_losses = []
    with torch.no_grad():
        for x, y in val_loader:
            x, y = x.to(device), y.to(device)
            pred = model(x).squeeze(-1)
            va_losses.append(loss_fn(pred, y).item())

    tr_loss = np.mean(tr_losses)
    va_loss = np.mean(va_losses)
    print(f"ep {epoch:03d} | tr {tr_loss:.4f} | va {va_loss:.4f}")

    if va_loss < best_val:
        best_val = va_loss
        torch.save(model.state_dict(), "best.pt")
```

- Classification αντί για regression: `y.long()`, `out_dim = num_classes`, `loss = CrossEntropyLoss()`, χωρίς `squeeze(-1)`.
- Χρήσιμα blocks: Linear, ReLU, Dropout, Conv2d, MaxPool2d, Embedding, LSTM, TransformerEncoderLayer.
- Losses: MSELoss, L1Loss, SmoothL1Loss, CrossEntropyLoss, BCEWithLogitsLoss.
- Optimizers: SGD, Adam, AdamW. Schedules: StepLR, CosineAnnealingLR, OneCycleLR.
- `model.train()` / `model.eval()` / `torch.no_grad()` πρέπει να ξέρεις ακριβώς πού μπαίνουν.

```
nn.Linear(in_features, out_features)
nn.ReLU()
nn.Dropout(p=0.2)
nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1)
nn.MaxPool2d(kernel_size=2, stride=2)
nn.Embedding(vocab_size, embed_dim)
nn.LSTM(in_size, hidden, num_layers=2, batch_first=True)

nn.MSELoss()
nn.CrossEntropyLoss()
nn.BCEWithLogitsLoss()

torch.optim.SGD(params, lr=0.1, momentum=0.9)
torch.optim.Adam(params, lr=1e-3)
torch.optim.AdamW(params, lr=1e-3, weight_decay=1e-4)
```

- Συνηθισμένα errors: size mismatch, Expected Long but got Float, Expected Float but got Double, CUDA out of memory, Loss = NaN.
- Tiny-batch overfit test: αν 8 δείγματα δεν πάνε σχεδόν στο 0 loss, κάτι είναι λάθος στο μοντέλο ή στη loss.

```
# Tiny-batch overfit test
tiny_x = Xtr[:8].to(device)
tiny_y = ytr[:8].to(device)

for step in range(200):
    pred = model(tiny_x).squeeze(-1)
    loss = loss_fn(pred, tiny_y)
    opt.zero_grad()
    loss.backward()
    opt.step()
```

6. Computer vision

Shape conventions που πρέπει να ξέρεις: NumPy/PIL -> (H, W, C), PyTorch -> (C, H, W), batches -> (N, C, H, W), cv2 -> BGR.

- Default κίνηση για μικρά image datasets: pretrained ResNet18 / EfficientNet / ConvNeXt.
- Πάγωσε backbone, άλλαξε classifier head, train λίγες εποχές, και μόνο αν προλαβαίνεις κάνε unfreeze τελευταίο block.
- Για pretrained ImageNet μοντέλα χρησιμοποίησε standard normalization.
- Simple CNN από το μηδέν μόνο όταν το domain είναι πολύ ιδιαίτερο.
- CLIP zero-shot όταν έχεις εικόνες + labels αλλά καθόλου training data.
- YOLO / DETR για detection μόνο αν το ζητάει ρητά η εκφώνηση.
- U-Net μόνο αν το ζητά segmentation.

```
from PIL import Image
import numpy as np
import torch

img = Image.open("cat.jpg").convert("RGB")
img = img.resize((224, 224))
arr = np.array(img)
arr_f = arr.astype(np.float32) / 255.0

t = torch.from_numpy(arr_f).permute(2, 0, 1)
t = t.unsqueeze(0)
img_back = t.squeeze(0).permute(1, 2, 0).cpu().numpy()
```

```
import torch, torch.nn as nn
from torchvision import models, transforms as T
from PIL import Image

device = "cuda" if torch.cuda.is_available() else "cpu"

feat_model = models.resnet18(weights="DEFAULT")
feat_model.fc = nn.Identity()
feat_model = feat_model.to(device).eval()

tf = T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize([0.485, 0.456, 0.406],
                [0.229, 0.224, 0.225]),
])

def extract_features(paths, bs=32):
    feats = []
    with torch.no_grad():
        for i in range(0, len(paths), bs):
            chunk = paths[i:i+bs]
            batch = torch.stack([
                tf(Image.open(p).convert("RGB")) for p in chunk
            ]).to(device)
            feats.append(feat_model(batch).cpu().numpy())
    return np.concatenate(feats, 0)
```

```
class SmallCNN(nn.Module):
    def __init__(self, in_ch=1, n_cls=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(in_ch, 32, 3, padding=1), nn.ReLU(),
            nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
            nn.Conv2d(64, 64, 3, padding=1), nn.ReLU(),
            nn.MaxPool2d(2),
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
        )
        self.clf = nn.Sequential(
```

```
        nn.Linear(64, 128), nn.ReLU(),
        nn.Dropout(0.3),
        nn.Linear(128, n_cls),
    )

    def forward(self, x):
        return self.clf(self.features(x))
```

7. NLP και embeddings

- TF-IDF είναι το γρήγορο, φθηνό και συχνά πολύ δυνατό baseline για text classification.
- Για noisy text / Greeklish, τα char n-grams είναι συχνά χρυσά.
- Όταν θες κάτι δυνατότερο από TF-IDF: embed τα κείμενα με pretrained encoder μία φορά και μετά βάλε sklearn πάνω στα vectors.
- Καλά multilingual defaults: multilingual-e5-small/base/large, MiniLM multilingual, xlm-roberta-base.
- Pooling strategy: mean pool ασφαλές default.
- Fine-tuning transformer μόνο αν το ζητάει ξεκάθαρα το πρόβλημα.
- Για retrieval / matching / duplicate detection: cosine similarity στα normalized embeddings.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from scipy.sparse import hstack
```

```
vec = TfidfVectorizer(
    ngram_range=(1, 2),
    min_df=3,
    max_df=0.95,
    sublinear_tf=True,
    max_features=200_000,
)

vec_char = TfidfVectorizer(
    analyzer="char_wb",
    ngram_range=(3, 5),
    min_df=3,
)

X_word = vec.fit_transform(texts)
X_char = vec_char.fit_transform(texts)
X = hstack([X_word, X_char]).tocsr()
```

```
import torch, numpy as np
from transformers import AutoTokenizer, AutoModel

device = "cuda" if torch.cuda.is_available() else "cpu"
MODEL = "intfloat/multilingual-e5-base"
tok = AutoTokenizer.from_pretrained(MODEL)
enc = AutoModel.from_pretrained(MODEL).to(device).eval()

def mean_pool(hidden, mask):
    m = mask.unsqueeze(-1).float()
    return (hidden * m).sum(1) / m.sum(1).clamp(min=1)

def embed(texts, batch=32, prefix="query: "):
    out = []
    for i in range(0, len(texts), batch):
        chunk = [prefix + t for t in texts[i:i+batch]]
        b = tok(chunk, padding=True, truncation=True,
                max_length=256, return_tensors="pt").to(device)
        with torch.no_grad():
            h = enc(**b).last_hidden_state
            emb = mean_pool(h, b.attention_mask)
        out.append(emb.cpu().numpy())
    return np.concatenate(out, 0)
```

```
from sklearn.preprocessing import normalize
from sklearn.linear_model import LogisticRegression

Xtr_e = normalize(embed(train_texts))
Xva_e = normalize(embed(val_texts))

clf = LogisticRegression(
    max_iter=2000,
    C=5.0,
    class_weight="balanced",
    n_jobs=-1,
).fit(Xtr_e, ytr)

print("acc:", clf.score(Xva_e, yva))
```

```
from sklearn.metrics.pairwise import cosine_similarity

A = normalize(embed(corpus))
q = embed([query])[0]
q = q / np.linalg.norm(q)

sims = A @ q
top5 = np.argsort(-sims)[:5]
```

8. Audio

- Δύο βασικά σενάρια: classification με embeddings ή transcription με Whisper.
- Σχεδόν ποτέ δεν σχεδιάζεις audio model από το μηδέν σε 4 ώρες.

```
import torchaudio
wav, sr = torchaudio.load("a.wav")

if sr != 16000:
    wav = torchaudio.functional.resample(wav, sr, 16000)
    sr = 16000

if wav.shape[0] > 1:
    wav = wav.mean(dim=0, keepdim=True)

import torchaudio.transforms as AT
mel = AT.MelSpectrogram(
    sample_rate=16000,
    n_fft=400,
    win_length=400,
    hop_length=160,
    n_mels=80,
)(wav)
log_mel = torch.log(mel + 1e-9)
```

```
from transformers import pipeline

asr = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-small",
    device=0 if torch.cuda.is_available() else -1,
    chunk_length_s=30,
    stride_length_s=5,
)

out = asr("a.wav")
print(out["text"])
```

```
from transformers import AutoFeatureExtractor, AutoModel

MODEL = "facebook/hubert-base-ls960"
feat = AutoFeatureExtractor.from_pretrained(MODEL)
enc = AutoModel.from_pretrained(MODEL).to(device).eval()

def audio_embed(wav, sr=16000):
    inputs = feat(wav.squeeze().numpy(),
                  sampling_rate=16000,
                  return_tensors="pt")
    inputs = {k: v.to(device) for k, v in inputs.items()}
    with torch.no_grad():
        h = enc(**inputs).last_hidden_state
    return h.mean(dim=1).squeeze().cpu().numpy()
```

9. Problem recipes

9.1 Embeddings -> regression / classification

- Μοτίβο: σου δίνουν vector ανά δείγμα (384/768/1024 dimensions).
- Αν το metric είναι MAE και το target ordinal 1..5, το απλό baseline είναι LightGBM με objective="mae".
- Συχνά καλό branch: classification πάνω στις κλάσεις και μετά argmin expected MAE cost.

```
import numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
from lightgbm import LGBMRegressor

X = np.load("train_embeddings.npy")
y = np.load("train_labels.npy").astype(float)

Xtr, Xva, ytr, yva = train_test_split(
    X, y, test_size=0.15, random_state=0
)

model = LGBMRegressor(
    objective="mae",
    n_estimators=1000,
    learning_rate=0.05,
    num_leaves=63,
    min_child_samples=20,
    verbose=-1,
)

model.fit(Xtr, ytr, eval_set=[(Xva, yva)])
pred = model.predict(Xva)
pred = np.clip(np.round(pred), 1, 5)

print("val MAE:", mean_absolute_error(yva, pred))
```

```
from sklearn.linear_model import LogisticRegression

y_int = y.astype(int)
classes = np.sort(np.unique(y_int))
y_idx = np.searchsorted(classes, y_int)

clf = LogisticRegression(
    max_iter=2000,
    C=1.0,
    class_weight="balanced",
    n_jobs=-1,
)
clf.fit(Xtr, y_idx[:len(Xtr)])

proba = clf.predict_proba(Xva)
costs = np.abs(classes[None, :] - classes[:, None])
exp_cost = proba @ costs
pred_idx = np.argmin(exp_cost, axis=1)
pred = classes[pred_idx]
```

9.2 Fixed μικρό MLP για target function

- Σημαντικότερο trick: Fourier positional encoding όταν επιτρέπεται.
- Κέρδη έρχονται από input representation, optimizer, schedule, loss και αριθμό βημάτων - όχι από αλλαγή architecture.

```
def fourier_encode(x, num_bands=8, max_freq=10.0):
    freqs = 2.0 ** torch.linspace(
        0, np.log2(max_freq), num_bands, device=x.device
    )
    x_proj = x.unsqueeze(-1) * freqs
    x_proj = x_proj.reshape(x.shape[0], -1) * np.pi
    return torch.cat([x, torch.sin(x_proj), torch.cos(x_proj)], dim=-1)
```

```
model = nn.Sequential(
    nn.Linear(in_dim, 10), nn.ReLU(),
```



```

nn.Linear(10, 10), nn.ReLU(),
nn.Linear(10, 10), nn.ReLU(),
nn.Linear(10, 10), nn.ReLU(),
nn.Linear(10, 10), nn.ReLU(),
nn.Linear(10, 10), nn.ReLU(),
nn.Linear(10, 3),
).to(device)

opt = torch.optim.Adam(model.parameters(), lr=1e-3)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=20000, eta_min=1e-5)
loss_fn = nn.MSELoss()

```

9.3 Codenames / greedy search

```

def l2norm(x):
    return x / (np.linalg.norm(x, axis=-1, keepdims=True) + 1e-9)

vocab_vecs = l2norm(vocab_vecs)
board_vecs = l2norm(board_vecs)

def score_clue(clue_idx):
    sims = board_vecs @ vocab_vecs[clue_idx]
    order = np.argsort(-sims)
    n_good = 0
    for b in order:
        if b in bad_idx:
            break
        if b in remaining:
            n_good += 1
    return n_good

```

9.4 Text / image / tabular / clustering / time series quick baselines

- Text: TF-IDF + LogisticRegression ή embeddings + LogisticRegression.
- Image: transfer learning με frozen backbone.
- Tabular: LightGBM με category dtype.
- Clustering: KMeans + silhouette / PCA / UMAP.
- Time series: sliding windows + LightGBM, time-based split.

```

# Text
vec = TfidfVectorizer(ngram_range=(1, 2), min_df=3, max_df=0.95)
clf = LogisticRegression(max_iter=2000, class_weight="balanced")

# Image
model = models.resnet18(weights="DEFAULT")
for p in model.parameters():
    p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, NUM_CLASSES)

# Tabular
for c in CAT_COLS:
    df[c] = df[c].astype("category")
model = LGBMRegressor(objective="mae", n_estimators=2000, learning_rate=0.03)

# Clustering
km = KMeans(n_clusters=best_k, n_init=20, random_state=0).fit(Xs)

# Time series
cut = int(len(df) * 0.85)
Xtr, Xva = X[:cut], X[cut:]
ytr, yva = y[:cut], y[cut:]

```

10. Optimization & solvers

- Όταν το πρόβλημα είναι assignment, scheduling, covering, subset selection ή άλλο discrete combinatorial choice, σκέψου solver.
- PuLP για linear / integer programming.
- OR-Tools CP-SAT για λογικούς περιορισμούς, all-different, bin packing, scheduling.
- z3 για feasibility / SAT-like constraints.

```
import pulp

N, M = 5, 3
cost = [[4, 2, 8],
        [1, 9, 3],
        [5, 4, 6],
        [7, 2, 3],
        [2, 8, 1]]

prob = pulp.LpProblem("assignment", pulp.LpMinimize)
x = [[pulp.LpVariable(f"x_{i}_{j}", cat="Binary")
      for j in range(M)] for i in range(N)]

prob += pulp.lpSum(cost[i][j] * x[i][j] for i in range(N) for j in range(M))

for i in range(N):
    prob += pulp.lpSum(x[i][j] for j in range(M)) == 1

for j in range(M):
    prob += pulp.lpSum(x[i][j] for i in range(N)) <= 2

prob.solve(pulp.PULP_CBC_CMD(msg=False))
```

```
from ortools.sat.python import cp_model

N = 8
model = cp_model.CpModel()
queens = [model.NewIntVar(0, N-1, f"q{i}") for i in range(N)]

model.AddAllDifferent(queens)
model.AddAllDifferent([queens[i] + i for i in range(N)])
model.AddAllDifferent([queens[i] - i for i in range(N)])

solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = 10
status = solver.Solve(model)
```

```
from z3 import Int, Solver, Or, sat

x, y, z = Int("x"), Int("y"), Int("z")
s = Solver()
s.add(x + y + z == 10)
s.add(x > 0, y > 0, z > 0)
s.add(Or(x == y, y == z))

if s.check() == sat:
    print(s.model())
```

11. Debugging & gotchas

Time management 4 ωρών: 0-15 λεπτά διάβασμα όλων, 15-60 baseline P1, 60-120 baseline P2, 120-180 baseline P3, 180-240 βελτίωση του πιο υποσχόμενου προβλήματος. Baselines πριν από tweaks.

- ValueError: inconsistent numbers of samples -> τα X και y έχουν διαφορετικό πρώτο dimension.
- could not convert string to float -> τάισες κείμενο σε numeric estimator.
- Input contains NaN -> κάνε imputation ή χρησιμοποίησε μοντέλο που χειρίζεται NaNs.
- classification metrics on continuous targets -> μπέρδεμα regressor/classifier.
- shapes not aligned -> λάθος matmul dimensions.
- KeyError σε DataFrame -> λάθος όνομα στήλης.
- CUDA out of memory -> μείωσε batch size.
- Expected Long but got Float -> CrossEntropy wants integer labels.
- Expected all tensors to be on the same device -> ξέχασες .to(device).
- Checklist όταν το μοντέλο δεν μαθαίνει: print shapes/dtypes, κοίτα samples, plot target distribution, random baseline, overfit one batch, LR x10 / LR /10, σωστή σειρά zero_grad -> backward -> step, loss-output match, leakage check.

```
# Order that must be correct
opt.zero_grad()
loss.backward()
opt.step()

# Useful prints
print(X.shape, y.shape, X.dtype, y.dtype)
print(y[:10])

# For classification
from sklearn.metrics import confusion_matrix, classification_report
print(confusion_matrix(y_true, y_pred))
print(classification_report(y_true, y_pred))
```